In this chapter, you'll see:

- How your specs can give you confidence in your code
- How a good test suite makes refactoring possible
- How to guide your design using your specs with behavior-driven development (BDD)

CHAPTER 3

The RSpec Way

Over the last couple of chapters, you've gotten to know RSpec. You've written your first few examples and organized them into groups. You've seen how to run just a filtered subset of your specs and how to customize output.

All these features of RSpec are designed to make certain habits easy:

- Writing examples that clearly spell out the expected behavior of the code
- Separating common setup code from the actual test logic
- Focusing on just what you need to do to make the next spec pass

None of these habits comes without a cost:

- Writing specs takes time.
- Running large suites takes time (or requires learning options to trim down the set).
- Reading heavily factored specs requires hopping between setup and test code.

We don't want anyone to take it for granted that the habits we're forming are good ones. We'd like to show you they're worth the cost. In this chapter, we're going to walk you through RSpec's approach to software development and what makes a good set of specs.

What Your Specs Are Doing for You

Writing specs isn't the goal of using RSpec—it's the benefits those specs provide. Let's talk about those benefits now; they're not all as obvious as "specs catch bugs."